MakeMyMarrige

DATABASE DESIGN SPECIFICATION

Version 1.0 | November 2026

MongoDB Atlas • Hybrid Modelling • Multi-tenant Workspace Architecture

1. Document control

• MakeMyMarrige Database Design Specification | Version 1.0 | November 2026

• Status: approved architecture baseline. Provider, legal, residency and exact retention verifications remain production prerequisites.

• Stack: Next.js + TypeScript PWA; NestJS REST API; MongoDB Atlas; managed Redis/BullMQ; Cloudflare R2. Initial launch: Kanpur, Uttar Pradesh, India. Workspace timezone: Asia/Kolkata.

2. Executive summary

• Hybrid MongoDB modelling: independently managed entities live in separate collections; only small, bounded, parent-owned values may be embedded. One collection per independently managed domain entity.

• User identity and account sessions are global. Wedding business data is tenant-owned and carries weddingId. The API derives tenant context from authenticated membership and never trusts a client-supplied weddingId as authority.

• ObjectId is used internally. Security-sensitive external flows use high-entropy opaque tokens, storing only token hashes. Money uses integer paise and INR. Date-only values use validated YYYY-MM-DD strings; actual instants use UTC BSON Date.

• Critical multi-document operations use MongoDB transactions. Asynchronous side effects use a transactional outbox and idempotent workers. Mutable records use optimistic concurrency control (OCC).

3. Approved modelling principles

• Reference independently changing, separately permissioned, high-cardinality entities. Embed only bounded child data sharing its parent's lifecycle and permissions.

• Keep histories, payments, audit events, notifications, guests, tasks and photos out of unbounded parent arrays.

• MongoDB references are not foreign keys. NestJS validates existence, same-tenant ownership, lifecycle eligibility and permissions.

• Use explicit lifecycle states instead of a universal isDeleted flag. Wedding deletion follows WF12.

• Redis is not the source of truth. R2 stores photo objects; MongoDB stores metadata and lifecycle state.

• Use DTO validation, domain validation and MongoDB JSON Schema validation. Use repository boundaries without hiding transactions, aggregation or conditional updates.

4. Tenant and ownership model

• Global collections: users, sessions, emailVerificationTokens, passwordResetTokens. Tenant-owned collections: memberships, invitations, guests, households, functions, tasks, RSVP, finance, vendors, Shagun, photos, notifications and wedding audit records. Platform/support records are separately restricted.

• Every tenant-owned document carries weddingId. NestJS resolves the wedding context from authenticated membership. A user may have only one ACTIVE or SUSPENDED wedding membership; SUSPENDED still occupies the slot.

• Wedding deletion must not delete global user accounts or unrelated account sessions. Account suspension, session revocation and wedding membership status are distinct.

5. Collection inventory

• Identity/global: users, sessions, emailVerificationTokens, passwordResetTokens.

• Workspace: weddings, weddingMemberships, invitations, membershipPeriodEvents.

• Governance: governanceRequests, weddingFinanceSettings, permissionChangeEvents.

• Planning: weddingFunctions, functionPublicationSnapshots, functionHouseholdInvitations, functionGuestInvitations, tasks, taskAssignments, taskChecklistItems, taskDependencies, taskComments, taskActivityEvents.

• Guests: households, guests, guestRelationshipTags, guestCorrectionRequests, guestMergeOperations, guestHouseholdHistory.

• RSVP: rsvps, rsvpEvents, additionalGuestRequests, additionalGuestRequestEvents.

• Finance: weddingBudgets, budgetCategories, budgetGovernanceRequests, budgetReserveAllocations, expenseEstimates, vendorQuotes, expenses, expenseApprovalEvents, expensePayments, expenseAdjustments, financeAuditEvents.

• Vendors: vendorReferences, weddingVendors, vendorCategories, vendorServices, vendorEnquiries, vendorEstimates, vendorAvailabilityChecks, vendorSelections, vendorBookingRecords, vendorActivityEvents.

• Shagun: shagunSettings, shagunConfigurationRequests, shagunPaymentIntents, shagunContributions, shagunProviderEvents, shagunReconciliationEvents, shagunAcknowledgements, shagunAuditEvents.

• Photos: photoAlbums, weddingPhotos, photoReports, photoRemovalRequests, photoAuditEvents. Notifications: notifications, notificationPreferences, optional dashboardSnapshots.

• Audit/lifecycle: auditEvents, securityIncidents, dataAccessRequests, dataCorrectionRequests, weddingLifecycleEvents, weddingDeletions, weddingDeletionTombstones, weddingExportRequests, cancellationNoticeRecords.

• Platform/infrastructure: supportRecoveryCases, outboxEvents, inboxEvents, idempotencyRecords, migrationHistory.

6. Relationship map

• User 1—N sessions and historical wedding memberships. Wedding 1—N memberships, invitations, guests, households, functions, tasks, expenses, vendors, photos, notifications and audit events.

• Household 1—N guests; each guest has at most one active household per wedding. Function 1—N RSVP records; unique RSVP key is (weddingId, functionId, guestId).

• Task 1—N assignments, checklist items, comments and activity events. Task dependencies are same-wedding and acyclic. Expense 1—N approval events, payment entries and adjustments.

• Vendor 1—N enquiries, estimates, quotes, availability checks, selection and booking records. Shagun contributions are separate from expenses. Photo albums contain photos; photos may have reports and removal requests.

7. Common field conventions

• _id and internal references: ObjectId. weddingId: ObjectId on tenant-owned records. createdAt/updatedAt: UTC BSON Date; append-only records omit updatedAt. status: uppercase SCREAMING_SNAKE_CASE. Mutable records use integer version for OCC.

• Money: amountPaise as integer/safe integer; currency:'INR'. Never use floating-point rupees for calculations. Display rupees using ceil(amountPaise/100); display rounding never changes actual amounts.

• Date-only: validated YYYY-MM-DD string, not a timestamp. Local time: validated HH:mm interpreted in Asia/Kolkata. Actual deadlines/instants: UTC BSON Date. Store tokenHash, never raw tokens.

• Validate calendar correctness, not merely date regex. Validate safe integer monetary bounds. Define explicit maximum lengths and array caps per schema.

8. Identity and workspace schemas

• users (global): _id, emailNormalized unique, passwordHash optional, emailVerifiedAt, displayName, accountStatus, bounded provider links, createdAt, updatedAt, version. Never embed sessions.

• sessions (global): _id, userId, refreshTokenHash, sessionFamilyId, createdAt, expiresAt, revokedAt, lastUsedAt, minimal device/IP metadata. TTL only under explicit lifecycle policy.

• emailVerificationTokens/passwordResetTokens (global): _id, userId, tokenHash unique, expiresAt, consumedAt, createdAt. TTL on expiresAt; reject expired/consumed tokens immediately.

• weddings: _id, name, weddingDate YYYY-MM-DD, timezone:'Asia/Kolkata', lifecycleStatus, deletionDueAt, createdAt, updatedAt, version.

• weddingMemberships: _id, weddingId, userId, memberType CO_OWNER|ORGANISER|TREASURER, status ACTIVE|SUSPENDED|ENDED, permissions, joinedAt, endedAt, version. Put potentially growing membership-period history in append-only membershipPeriodEvents.

• invitations: _id, weddingId, invitedEmailNormalized, invitationType, tokenHash unique, status PENDING|ACCEPTED|REVOKED|EXPIRED, expiresAt, createdBy, acceptedBy, acceptedAt, createdAt. Retain history; no TTL.

• governanceRequests: _id, weddingId, requestType, targetEntityType/Id, bounded immutable proposedSnapshot, proposalVersion, distinct co-owner approvals, status, expiresAt, createdBy, createdAt, version.

9. Guests, functions and RSVP schemas

• weddingFunctions: _id, weddingId, name, date YYYY-MM-DD, startTime/endTime HH:mm, venueVisibility, draftDetails, publicationStatus, version, timestamps.

• functionPublicationSnapshots: _id, weddingId, functionId, snapshotVersion, exactly two co-owner approvals, immutable publishedSnapshot, publishedAt. Unique weddingId+functionId+snapshotVersion.

• households: _id, weddingId, label, primaryContactGuestId, status ACTIVE|ARCHIVED, timestamps, version. Primary contact must be active in same household.

• guests: _id, weddingId, householdId, fullName, normalised phone/email, status ACTIVE|ARCHIVED, verification state, relationship tag references, timestamps, version.

• guestRelationshipTags: _id, weddingId, name, tagType PRIMARY|SECONDARY, active. Enforce one active primary assignment per guest in domain logic.

• guestCorrectionRequests: _id, weddingId, guestId, requestedChanges, requester, status, reviewer, reason, timestamps. Requester cannot approve own request.

• guestMergeOperations: _id, weddingId, sourceGuestId, targetGuestId, requester, two co-owner approvals, status, timestamps. Preserve historical references.

• guestHouseholdHistory: _id, weddingId, guestId, fromHouseholdId, toHouseholdId, changedBy, reason, changedAt; append-only.

• functionHouseholdInvitations/functionGuestInvitations: _id, weddingId, functionId, householdId or guestId, status, invitedAt, createdBy. New household members are not auto-invited.

• rsvps: _id, weddingId, functionId, guestId, response NOT_YET_RESPONDED|ATTENDING|NOT_ATTENDING, respondedAt, source, version, timestamps. Unique weddingId+functionId+guestId.

• rsvpEvents: _id, weddingId, rsvpId, guestId, functionId, previous/new response, actor, reason, occurredAt; append-only.

• additionalGuestRequests: _id, weddingId, functionId, requestingGuestId, requestedCount, approvedCount, status, timestamps, version. Events stored separately.

10. Tasks and planning schemas

• tasks: _id, weddingId, optional functionId, title, description, status, priority, dueAt, createdBy, version, timestamps.

• taskAssignments: _id, weddingId, taskId, assigneeMembershipId/userId, assignedBy, assignedAt, status. Unique active task/assignee pair.

• taskChecklistItems: _id, weddingId, taskId, title, isMandatory, status OPEN|COMPLETED, completedBy, completedAt, sortOrder, version. Separate collection because item completion and mandatory progress are independently validated.

• taskDependencies: _id, weddingId, taskId, dependsOnTaskId, createdBy, createdAt. Unique edge; reject self-reference and cycles.

• taskComments: _id, weddingId, taskId, authorId, body, createdAt, optional audited edit metadata. taskActivityEvents are append-only.

11. Finance schemas

• weddingBudgets: _id, weddingId unique, totalBudgetPaise, reservePaise, currency INR, version, updatedAt, updatedBy. Reserve is inside total budget.

• budgetCategories: _id, weddingId, name, allocatedPaise, optional rebuildable aggregates, sortOrder, status, version.

• budgetGovernanceRequests: _id, weddingId, requestType, source/target category, amountPaise, proposalSnapshot, approvals, status, timestamps, version. budgetReserveAllocations is append-only transfer history.

• expenseEstimates: _id, weddingId, categoryId, optional vendorId, low/expected/high paise, assumptions, creator, timestamps, version.

• expenses: _id, weddingId, categoryId, optional vendorId, title, beneficiary, creator, amountPaise, currency INR, lifecycle status, approval metadata, version, timestamps.

• expenseApprovalEvents: _id, weddingId, expenseId, action, actor, reason, approvalRoute, occurredAt; append-only.

• expensePayments: _id, weddingId, expenseId, amountPaise, currency INR, paidAt, recordedBy, method, reference, optional evidence object key, createdAt. No silent edits/deletes.

• expenseAdjustments: _id, weddingId, expenseId, adjustmentType, amountPaise, reason, requester, approvals, status, timestamps.

• financeAuditEvents: _id, weddingId, entityType/Id, action, actor, before/after summary, reason, occurredAt. Treasurer-created or Treasurer-beneficiary expenses require both co-owners; normal expense approval is Treasurer-only.

12. Vendor and Shagun schemas

• vendorReferences: _id, provider GOOGLE_PLACES|MANUAL, providerPlaceId, displayName, category, locationSummary, sourceMetadata, timestamps. A listing is not a booking.

• weddingVendors: _id, weddingId, vendorReferenceId, displayNameSnapshot, categoryId, source, status, creator, timestamps, version.

• vendorCategories/services: wedding-scoped category/service records; categories declare whether multiple selections are allowed.

• vendorEnquiries: _id, weddingId, vendorId, status NOT_CONTACTED|CONTACTED|RESPONSE_RECEIVED|NEGOTIATING|SELECTED|REJECTED, manager, notes, timestamps, version.

• vendorEstimates/vendorQuotes: weddingId, vendorId, estimate ranges or quote amount/lines, inclusions, exclusions, taxes, terms, validity and status. Quote/selection is not proof of booking.

• vendorAvailabilityChecks: explicit state NOT_CHECKED|AVAILABLE_CONFIRMED|UNAVAILABLE_CONFIRMED|TENTATIVE|UNKNOWN, checkedBy, checkedAt. Never infer availability.

• vendorSelections/vendorBookingRecords: explicit selection and booking lifecycle; enforce one active selection per category unless multiple are allowed.

• shagunSettings: _id, weddingId unique, providerId, encrypted destination reference, masked display, status, version. One UPI destination per wedding.

• shagunConfigurationRequests: proposed encrypted destination, two co-owner approvals, status and timestamps.

• shagunPaymentIntents: _id, weddingId, guestId, providerId, providerIntentId, amountPaise, INR, status, idempotencyKey, timestamps, version.

• shagunContributions: _id, weddingId, guestId, paymentIntentId, providerPaymentId, amountPaise, INR, verifiedAt, status. Only provider-verified status counts as success.

• shagunProviderEvents: unique providerId+providerEventId, payloadHash, receivedAt, processing status. Reconciliation events, acknowledgements and audit events remain separate.

13. Photos, notifications, audit and lifecycle schemas

• photoAlbums: _id, weddingId, functionId, title, createdAt, version. Unique weddingId+functionId.

• weddingPhotos: _id, weddingId, albumId, functionId, uploader, private R2 object/derivative keys, MIME type, size, dimensions, checksum, moderation status, exifStripped, timestamps, version. No permanent public URLs.

• photoReports/photoRemovalRequests: separate records with reporter/requester, reason, status, reviewer and timestamps. photoAuditEvents append-only.

• notifications: _id, weddingId, recipient membership/user, category, severity, title, body summary, deep link, readAt, createdAt, dedupeKey. Recheck permission when opening deep links.

• notificationPreferences: _id, weddingId, recipientMembershipId, category preferences, updatedAt, version. Optional dashboardSnapshots are rebuildable and never authoritative.

• auditEvents: _id, weddingId, actor, action, entity type/id, requestId, reason, before/after summary, occurredAt. Application append-only; sensitive reads/exports recorded.

• securityIncidents: _id, weddingId, severity SEV_1..SEV_4, lifecycle status, summary, owner, timestamps, version. Data access/correction requests separate.

• weddingLifecycleEvents, weddingDeletions, weddingDeletionTombstones, weddingExportRequests and cancellationNoticeRecords are separate. Deletion status: QUEUED|IN_PROGRESS|RETRY_PENDING|BLOCKED|COMPLETED|FAILED_REQUIRES_REVIEW.

• supportRecoveryCases are platform-level, tightly restricted and audited.

14. Infrastructure collections

• outboxEvents: stable event ID, optional weddingId, aggregate type/id, event type, payload version, bounded payload, status, occurredAt, publishedAt, attempt count, nextAttemptAt. Insert in same transaction as business write.

• inboxEvents: consumer name, source/provider, external event ID, payload hash, received/processed timestamps, status. Unique consumerName+externalEventId.

• idempotencyRecords: scope, actorId, idempotencyKey, requestHash, status, response reference, timestamps. Unique scope+actorId+key. Financial idempotency retention follows financial lifecycle.

• migrationHistory: unique migrationId, checksum, status STARTED|COMPLETED|FAILED, start/completion timestamps, executor and error summary.

15. Critical indexes

• users: unique emailNormalized.

• weddingMemberships: partial unique userId where status is ACTIVE or SUSPENDED; weddingId+status+memberType for workspace listings. Verify partialFilterExpression support and index build behaviour on selected MongoDB version.

• invitations: weddingId+invitedEmailNormalized+status; unique tokenHash.

• rsvps: unique weddingId+functionId+guestId; weddingId+functionId+response for counts.

• guests: weddingId+householdId+status; add normalised search index only for measured query patterns.

• tasks: weddingId+status+dueAt+_id; weddingId+functionId+status.

• expenses: weddingId+status+createdAt descending+_id descending; weddingId+categoryId+status.

• expensePayments: weddingId+expenseId+paidAt descending+_id descending.

• shagunProviderEvents: unique providerId+providerEventId; weddingId+processingStatus+receivedAt.

• weddingPhotos: weddingId+albumId+moderationStatus+createdAt descending+_id descending.

• notifications: weddingId+recipientMembershipId+readAt+createdAt descending+_id descending.

• auditEvents: weddingId+occurredAt descending+_id descending; weddingId+entityType+entityId+occurredAt descending.

• outboxEvents: status+nextAttemptAt+occurredAt; unique stable eventId. idempotencyRecords: unique scope+actorId+idempotencyKey.

• Tenant-first is a baseline, not a reason for redundant indexes. Compound indexes beginning with weddingId may satisfy tenant-prefix access. Validate using explain executionStats.

16. TTL policy

• TTL on emailVerificationTokens and passwordResetTokens expiresAt; application rejects expiry immediately.

• TTL on persisted OTP challenges expiresAt if used; OTP remains single-use and rate-limited.

• Temporary upload reservations may use TTL only with a cleanup worker that checks completion and reconciles R2 objects.

• No TTL on invitations, audit events, payment records, RSVP history, expense history or workspace deletion records.

• MongoDB TTL cleanup is asynchronous. It is physical cleanup, not a security or exact-time scheduling mechanism.

17. Transaction boundaries

• First co-owner onboarding: create wedding, first membership, audit/outbox in one transaction.

• Second co-owner acceptance: validate token/email and global membership slot; create/reactivate membership, accept invitation and write audit/outbox in one transaction.

• Governance approval: record distinct approvals; apply final transition and outbox event transactionally.

• Function publication: validate dual approval, write immutable snapshot, update function state and outbox atomically.

• RSVP mutation: conditional update + RSVP event + outbox event when required.

• Expense approval: validate approval route/no self-approval; conditional status/version update + approval event + outbox.

• Payment webhook: verify signature at ingress, deduplicate provider event, update intent/contribution and create outbox event transactionally.

• Photo moderation: conditional status update + audit/outbox. Workspace deletion: freeze/revoke and enqueue idempotent steps.

• Transactions must be short. Never call payment, email, Google Places or R2 APIs inside a MongoDB transaction.

18. Concurrency and idempotency

• Mutable documents carry integer version. Updates filter by _id, trusted weddingId and expected version, then increment version atomically. Stale version returns HTTP 409.

• State transitions use conditional updates including expected current status and version.

• Use unique constraints for RSVP identity, provider event identity, active assignments and idempotency keys.

• Webhook handlers tolerate duplicate and out-of-order delivery; reconcile against provider state where required.

• BullMQ jobs are at-least-once in practice; every worker must be safe to retry.

• Reject reuse of an idempotency key with a different request hash. Never perform external network calls inside a MongoDB transaction.

19. Query and pagination standards

• Every tenant query includes server-resolved weddingId and permission filters.

• Large collections use cursor pagination, normally createdAt DESC and _id DESC. Initial maximum page size: 50.

• Cursor payloads are opaque and validated; client cannot choose arbitrary sort fields. Cursor pagination is not snapshot-consistent under concurrent writes.

• Small bounded configuration lists may be unpaginated. Avoid unrestricted regex searches and deep skip offsets.

• Use explain executionStats for guest, RSVP, task, expense, photo, audit and dashboard query paths.

20. Security and privacy

• TLS in transit, managed database encryption at rest, least-privilege database identities and separate environment credentials.

• Secrets in managed secret storage; never commit credentials. Passwords use modern adaptive hashing; opaque tokens are hashed before storage.

• Use application-level encryption for selected sensitive fields when threat model requires it; define key management, rotation and recovery before production.

• Phone lookup may use normalised phone plus keyed HMAC lookup value where plaintext search is not required.

• Redact tokens, OTPs, passwords, payment credentials and sensitive personal data from logs and audit payloads.

• Re-authorise photo reads/downloads and notification deep links. Never expose permanent public R2 URLs.

21. Migrations and deployment

• Use ordered migration files with unique IDs and checksums; track execution in migrationHistory.

• Run migrations as a controlled single deployment job, not from every API instance on startup.

• Use expand-and-contract: add field, support transition, backfill, verify, switch reads/writes, remove old field in a later release.

• Test migrations against production-like data and edge cases. Keep local, test, staging and production databases isolated.

22. Backup, deletion and recovery

• Select Atlas tier/region only after verifying backup, point-in-time restore, transaction and residency capabilities. Define RPO/RTO only after testing.

• Restore drills must replay deletion tombstones so deleted workspaces do not reappear.

• Workspace deletion: freeze writes, revoke access, stop jobs, delete R2 objects, delete tenant records, request provider cleanup where supported, verify, then retain minimal tombstone.

• Backup retention must be bounded and documented. Legal/provider retention exceptions must be explicit and disclosed.

23. Observability

• Monitor query latency, slow queries, index usage, collection/index growth, connections, replication, storage and transaction aborts/retries.

• Monitor duplicate-key errors, OCC conflicts, outbox backlog, worker retries, TTL lag and deletion failures.

• Propagate request/correlation IDs through API, outbox and workers.

• Alert on financial reconciliation failures, webhook failures, blocked deletion jobs and unusual cross-tenant access denials.

24. Testing requirements

• Schema and MongoDB validator tests; repository integration tests against replica-set-capable MongoDB.

• Unique/partial index tests including concurrent insert races; tenant-isolation tests across repositories and APIs.

• Transaction rollback/retry tests; OCC lost-update tests; concurrent invitation acceptance and governance approval tests.

• Duplicate/out-of-order payment webhook and idempotency tests; migration tests; deletion and backup-restore tombstone tests.

• Load tests for guest directory, RSVP counts, photo gallery, expense lists and dashboard aggregation.

25. Workflow traceability

• WF01: users, weddings, weddingMemberships, invitations, emailVerificationTokens, sessions, auditEvents, outboxEvents.

• WF02: weddingMemberships, governanceRequests, weddingFinanceSettings, auditEvents, outboxEvents.

• WF03: weddingFunctions, functionPublicationSnapshots, functionHouseholdInvitations, functionGuestInvitations, tasks, taskAssignments, taskChecklistItems, taskDependencies, taskComments, taskActivityEvents.

• WF04: households, guests, guestRelationshipTags, guestCorrectionRequests, guestMergeOperations, guestHouseholdHistory.

• WF05: rsvps, rsvpEvents, additionalGuestRequests, additionalGuestRequestEvents, functionGuestInvitations.

• WF06: weddingBudgets, budgetCategories, budgetGovernanceRequests, budgetReserveAllocations, expenseEstimates, vendorQuotes, expenses, expenseApprovalEvents, expensePayments, expenseAdjustments, financeAuditEvents.

• WF07: vendorReferences, weddingVendors, vendorCategories, vendorServices, vendorEnquiries, vendorEstimates, vendorQuotes, vendorAvailabilityChecks, vendorSelections, vendorBookingRecords, vendorActivityEvents.

• WF08: shagunSettings, shagunConfigurationRequests, shagunPaymentIntents, shagunContributions, shagunProviderEvents, shagunReconciliationEvents, shagunAcknowledgements, shagunAuditEvents.

• WF09: photoAlbums, weddingPhotos, photoReports, photoRemovalRequests, photoAuditEvents.

• WF10: notifications, notificationPreferences, optional dashboardSnapshots, outboxEvents.

• WF11: auditEvents, sessions, securityIncidents, dataAccessRequests, dataCorrectionRequests, supportRecoveryCases.

• WF12: weddingLifecycleEvents, weddingDeletions, weddingDeletionTombstones, weddingExportRequests, cancellationNoticeRecords, supportRecoveryCases.

26. Open items before production

• Verify Atlas tier, region, transactions, backups, point-in-time recovery, encryption and residency.

• Confirm applicable Indian privacy and financial-record retention obligations with qualified legal/compliance advice.

• Select key management/rotation provider if application-level field encryption is used.

• Select UPI provider and verify beneficiary flow, webhook signatures, idempotency, reconciliation, settlement, reversal and retention.

• Verify R2 region/residency commitments; do not claim India-only residency without contractual evidence.

• Finalise OTP provider and challenge persistence. Define exact enums, field bounds, array caps and JSON Schema validators during implementation.

• Keep membership-period history in a separate append-only collection if it can grow; do not embed an unbounded history array.

27. Implementation sequence

• 1. Establish database module, connection policy, validators, migration runner and shared conventions.

• 2. Implement users, sessions, weddings, memberships and invitations; add tenant-isolation and concurrency tests.

• 3. Implement guests, households, functions, invitations and RSVP.

• 4. Implement governance, tasks, audit and outbox infrastructure.

• 5. Implement budget, expenses, vendors and Shagun after provider decisions.

• 6. Implement photo metadata, R2 integration and moderation.

• 7. Implement notifications, lifecycle deletion, backup-restore tombstones and operational dashboards.

• Introduce collections alongside their owning domain modules; do not deploy one giant migration.

Appendix A. Important qualifications

• MongoDB does not enforce relational foreign keys. Same-tenant reference integrity is an application responsibility.

• TTL deletion is asynchronous. Application logic must reject expired records immediately.

• Partial unique index syntax for ACTIVE/SUSPENDED memberships must be validated against the selected MongoDB server version and supported partial filter expressions.

• Unique indexes cannot enforce every cross-document invariant. Use transactions and domain services for primary-contact validity, task dependency acyclicity and similar rules.

• Financial totals and Shagun success must derive from authoritative records, not client claims or cached projections.

• This is the logical and implementation specification. Exact Mongoose code, final index scripts, DTOs and validators are codebase artifacts to implement and test.